Cámara en el registro

Esta guía abre la cámara del teléfono desde el registro. Al pulsar **Tomar foto**, Android muestra su diálogo de permiso la primera vez y después abre la aplicación de cámara del sistema. Si la persona acepta la imagen, el formulario la muestra encima de los campos.

La foto es opcional. El nombre, el correo, los términos, DataStore y las animaciones del botón **Registrarse** se quedan como están. La ruta `register_page` y el `NavHost` no se tocan.

1. Dependencias

En `app/build.gradle.kts`, dentro de `dependencies`, deben estar estas líneas. En este proyecto ya están. Si falta alguna, agrégala y sincroniza Gradle.

```
implementation(libs.androidx.activity.compose)
implementation(libs.androidx.core.ktx)
implementation(platform(libs.androidx.compose.bom))
implementation(libs.androidx.compose.ui)
implementation(libs.androidx.compose.ui.graphics)
```

Las versiones están en `gradle/libs.versions.toml`: `activity-compose` 1.13.0, `core-ktx` 1.19.0 y el BOM de Compose `2026.02.01`.

- `activity-compose` trae `rememberLauncherForActivityResult`, `TakePicture` y `RequestPermission`. Con eso la pantalla abre la cámara del sistema y el diálogo de permiso.
- `core-ktx` trae `FileProvider` y `ContextCompat`. El primero entrega la dirección del archivo. El segundo consulta si el permiso ya está concedido.
- `compose.ui` y `compose.ui.graphics` traen `Image` y `asImageBitmap()`. El BOM fija versiones compatibles entre esas librerías.

El permiso `CAMERA`, `Uri` y `BitmapFactory` vienen con el SDK de Android. Esta guía no usa `CameraX`.

2. El manifiesto

En `app/src/main/AndroidManifest.xml`, antes de `<application>`, declara el permiso y que la cámara no es obligatoria para instalar la app:

```
<uses-permission android:name="android.permission.CAMERA" />

<uses-feature
    android:name="android.hardware.camera"
    android:required="false" />
```

`uses-permission` anuncia que la app puede pedir la cámara. El permiso se concede después, cuando la persona pulsa el botón. `required="false"` deja instalar el registro en un aparato sin cámara.

Dentro de `<application>`, antes de `<activity>`, agrega el proveedor de archivos:

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths" />
</provider>
```

`FileProvider` convierte un archivo privado de la app en una dirección `content://`. La cámara escribe en esa dirección. `exported="false"` impide que otras apps abran el proveedor por su cuenta. `grantUriPermissions="true"` permite entregar un permiso temporal solo para esa foto. `${applicationId}` vale `cl.duoc.myapplication`, así que la autoridad queda `cl.duoc.myapplication.fileprovider`. El `ViewModel` usa ese mismo texto.

El manifiesto queda así:

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.CAMERA" />

    <uses-feature
        android:name="android.hardware.camera"
        android:required="false" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication">
        <provider
            android:name="androidx.core.content.FileProvider"
            android:authorities="${applicationId}.fileprovider"
            android:exported="false"
            android:grantUriPermissions="true">
            <meta-data
                android:name="android.support.FILE_PROVIDER_PATHS"
                android:resource="@xml/file_paths" />
        </provider>

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.MyApplication"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

3. Las rutas que la cámara puede escribir

Crea `app/src/main/res/xml/file_paths.xml`:

```xml
<?xml version="1.0" encoding="utf-8"?>
<paths>
    <cache-path
        name="fotos"
        path="fotos/" />
</paths>
```

`cache-path` apunta a la carpeta de caché de la app. `path="fotos/"` autoriza solo la subcarpeta `fotos`, que el `ViewModel` va a crear. `name="fotos"` es el tramo que aparece en la dirección

`content://`.

4. La dirección en la ficha

En `RegistroUiState`, agrega `fotoUri` al final:

```
data class RegistroUiState(
    val nombre: String = "",
    val correo: String = "",
    val aceptaTerminos: Boolean = false,
    val errores: RegistroErrores = RegistroErrores(),
    val guardando: Boolean = false,
    val guardado: Boolean = false,
    val fotoUri: String = ""
)
```

`fotoUri` es el texto de la dirección `content://`. Empieza vacío porque al abrir el registro todavía no hay foto. La imagen en sí sigue siendo un archivo. La ficha solo recuerda dónde está.

5. Crear el archivo

En `RegistroViewModel`, agrega estas dos funciones debajo de `guardar()`:

```
fun crearFoto(): Uri {
    val aplicacion = getApplication<Application>()
    val directorio = File(aplicacion.cacheDir, "fotos")
    if (!directorio.exists()) {
        directorio.mkdirs()
    }
    val archivo = File(directorio, "foto-${System.currentTimeMillis()}.jpg")
    archivo.createNewFile()
    return FileProvider.getUriForFile(
        aplicacion,
        "cl.duoc.myapplication.fileprovider",
        archivo
    )
}

fun onFotoTomada(uri: String) {
    _estado.update { it.copy(fotoUri = uri) }
}
```

Imports:

```
import android.net.Uri
import androidx.core.content.FileProvider
import java.io.File
```

`crearFoto` arma un JPEG vacío en `cache/fotos`, con la hora en el nombre para que cada toma sea un archivo distinto. `FileProvider.getUriForFile` devuelve la dirección que la cámara puede usar. La autoridad tiene que coincidir con la del manifiesto.

`onFotoTomada` publica esa dirección en la ficha. Se llama solo cuando la cámara responde que la foto quedó guardada. Si la persona cancela, esta función no corre y `fotoUri` sigue con el valor

anterior.

`RegistroPreferences` no cambia. El nombre y el correo siguen en DataStore. La foto vive en la caché mientras el `ViewModel` recuerda la dirección. Al rotar el teléfono, de vertical a horizontal, el `ViewModel` sigue vivo y la imagen se mantiene. Al cerrar la app, la caché puede desaparecer.

6. La pantalla

`RegisterScreen` pide el permiso, abre la cámara y dibuja la foto. Los contratos se crean en la composición, junto al `estado`, porque `rememberLauncherForActivityResult` tiene que ejecutarse cada vez que Compose arma la función, y el clic solo los dispara.

6.1 Leer el archivo para mostrarlo

Al final de `RegisterScreen.kt`, después de la función de la pantalla, agrega:

```
private fun cargarFoto(contexto: Context, uri: String): ImageBitmap? {
    val opciones = BitmapFactory.Options().apply { inSampleSize = 4 }
    val bitmap = contexto.contentResolver.openInputStream(Uri.parse(uri))?.use { entrada ->
        BitmapFactory.decodeStream(entrada, null, opciones)
    } ?: return null
    return bitmap.asImageBitmap()
}
```

Imports de esta función:

```
import android.content.Context
import android.graphics.BitmapFactory
import android.net.Uri
import androidx.compose.ui.graphics.ImageBitmap
import androidx.compose.ui.graphics.asImageBitmap
```

La cámara entrega una imagen grande. `inSampleSize = 4` toma un píxel de cada cuatro en cada lado, y la foto cabe en la pantalla. `openInputStream` abre la dirección `content://`. `asImageBitmap()` la convierte al tipo que `Image` de Compose sabe dibujar.

6.2 Permiso y cámara

Dentro de `RegisterScreen`, debajo de `paddingLogo`, agrega el contexto, la dirección pendiente y los dos lanzadores:

```
val contexto = LocalContext.current
var uriPendiente by remember { mutableStateOf<Uri?>(null) }

val camaraLauncher = rememberLauncherForActivityResult(
    ActivityResultContracts.TakePicture()
) { guardada ->
    val pendiente = uriPendiente
    if (guardada && pendiente != null) {
        viewModel.onFotoTomada(pendiente.toString())
    }
    uriPendiente = null
}

fun abrirCamara() {
    val uri = viewModel.crearFoto()
    uriPendiente = uri
    camaraLauncher.launch(uri)
}

val permisoLauncher = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestPermission()
) { concedido ->
    if (concedido) {
        abrirCamara()
    } else {
        scope.launch {
            snackbarHostState.showSnackbar("Sin permiso de cámara no se puede tomar la foto")
        }
    }
}

fun pedirFoto() {
    val tieneCamara =
        contexto.packageManager.hasSystemFeature(PackageManager.FEATURE_CAMERA_ANY)
    if (!tieneCamara) {
        scope.launch {
            snackbarHostState.showSnackbar("Este aparato no tiene cámara")
        }
        return
    }
    val permitido = ContextCompat.checkSelfPermission(
        contexto,
        Manifest.permission.CAMERA
    ) == PackageManager.PERMISSION_GRANTED
    if (permitido) {
        abrirCamara()
    } else {
        permisoLauncher.launch(Manifest.permission.CAMERA)
    }
}

val foto = remember(estado.fotoUri) {
    if (estado.fotoUri.isBlank()) {
        null
    } else {
        cargarFoto(contexto, estado.fotoUri)
    }
}
```

Imports:

```
import android.Manifest
import android.content.pm.PackageManager
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import androidx.compose.ui.platform.LocalContext
import androidx.core.content.ContextCompat
```

`uriPendiente` guarda la dirección mientras la cámara está abierta. Todavía no es la foto del formulario.

`TakePicture` es el contrato: recibe un `Uri`, abre la cámara del sistema y responde `true` si la persona aceptó la imagen. El contrato agrega el permiso temporal de escritura sobre esa dirección. `camaraLauncher.launch(uri)` es la llamada que abre la cámara.

`RequestPermission` abre el diálogo del sistema. Si la persona acepta, `concedido` es `true` y se llama a `abrirCamara()`. Si rechaza, el snackbar avisa y el formulario sigue disponible.

`pedirFoto` es lo que corre el botón. Primero mira si el aparato tiene cámara. Después mira si el permiso ya está concedido. `checkSelfPermission` compara con `PERMISSION_GRANTED`.

`foto` lee el archivo solo cuando `fotoUri` cambia. Mientras la dirección sigue vacía, vale `null` y la pantalla no dibuja imagen.

6.3 El botón y la imagen

La columna ahora es más larga que la pantalla. Sustituye el `modifier` de la `Column` para que se pueda desplazar, y quita los dos `Spacer` con `weight`:

```
Column(
    modifier = Modifier
        .fillMaxSize()
        .verticalScroll(rememberScrollState())
        .padding(16.dp),
    horizontalAlignment = Alignment.CenterHorizontally
) {
```

Debajo del logo, antes del campo nombre, muestra la foto y el botón:

```
if (foto != null) {
    Image(
        bitmap = foto,
        contentDescription = "Foto de registro",
        modifier = Modifier.size(120.dp),
        contentScale = ContentScale.Crop
    )
}

Button(onClick = { pedirFoto() }) {
    Text(if (estado.fotoUri.isBlank()) "Tomar foto" else "Tomar otra foto")
}
```

`Image` con `bitmap` dibuja la foto ya cargada. `size(120.dp)` la deja chica, encima de los campos. `Crop` recorta los bordes para llenar ese cuadrado. El texto del botón pasa a “Tomar otra foto” cuando ya hay una dirección guardada.

Cómo queda RegisterScreen

Parte 1, hasta el logo y la foto:

```
package cl.duoc.myapplication.ui

import android.Manifest
import android.content.Context
import android.content.pm.PackageManager
import android.graphics.BitmapFactory
import android.net.Uri
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateDpAsState
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Checkbox
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.derivedStateOf
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.ImageBitmap
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import androidx.core.content.ContextCompat
import androidx.lifecycle.viewmodel.compose.viewModel
import cl.duoc.myapplication.R
import cl.duoc.myapplication.navigation.Screen
import cl.duoc.myapplication.viewmodels.MainViewModel
import cl.duoc.myapplication.viewmodels.RegistroViewModel
import kotlinx.coroutines.launch

@Composable
fun RegisterScreen(mainViewModel: MainViewModel) {
    val viewModel: RegistroViewModel = viewModel()
    val estado by viewModel.estado.collectAsState()
```

```
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()
    var intentos by rememberSaveable { mutableStateOf(0) }
    val puedeEnviar by remember {
        derivedStateOf {
            estado.nombre.isNotBlank() &&
                estado.correo.contains("@") &&
                estado.aceptaTerminos
        }
    }
    val colorBoton by animateColorAsState(
        targetValue = when {
            estado.guardando -> Color.Gray
            estado.guardado -> Color(0xFF2E7D32)
            puedeEnviar -> MaterialTheme.colorScheme.primary
            else -> Color(0xFF9E9E9E)
        },
        label = "colorBoton"
    )
    val paddingLogo by animateDpAsState(
        targetValue = if (estado.guardado) 8.dp else 40.dp,
        label = "paddingLogo"
    )
    val contexto = LocalContext.current
    var uriPendiente by remember { mutableStateOf<Uri?>(null) }

    val camaraLauncher = rememberLauncherForActivityResult(
        ActivityResultContracts.TakePicture()
    ) { guardada ->
        val pendiente = uriPendiente
        if (guardada && pendiente != null) {
            viewModel.onFotoTomada(pendiente.toString())
        }
        uriPendiente = null
    }

    fun abrirCamara() {
        val uri = viewModel.crearFoto()
        uriPendiente = uri
        camaraLauncher.launch(uri)
    }

    val permisoLauncher = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { concedido ->
        if (concedido) {
            abrirCamara()
        } else {
            scope.launch {
                snackbarHostState.showSnackbar("Sin permiso de cámara no se puede tomar la
    foto")
            }
        }
    }

    fun pedirFoto() {
        val tieneCamara =
        contexto.packageManager.hasSystemFeature(PackageManager.FEATURE_CAMERA_ANY)
        if (!tieneCamara) {
            scope.launch {
                snackbarHostState.showSnackbar("Este aparato no tiene cámara")
            }
            return
        }
```

```
        val permitido = ContextCompat.checkSelfPermission(
            contexto,
            Manifest.permission.CAMERA
        ) == PackageManager.PERMISSION_GRANTED
        if (permitido) {
            abrirCamara()
        } else {
            permisoLauncher.launch(Manifest.permission.CAMERA)
        }
    }

    val foto = remember(estado.fotoUri) {
        if (estado.fotoUri.isBlank()) {
            null
        } else {
            cargarFoto(contexto, estado.fotoUri)
        }
    }

    Box {
        Image(
            painterResource(R.drawable.images),
            contentDescription = "",
            modifier = Modifier.fillMaxSize(),
            contentScale = ContentScale.Crop
        )

        Column(
            modifier = Modifier
                .fillMaxSize()
                .verticalScroll(rememberScrollState())
                .padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Image(
                painter = painterResource(R.drawable.logo),
                contentDescription = "Logo App",
                modifier = Modifier.padding(paddingLogo)
            )

            if (foto != null) {
                Image(
                    bitmap = foto,
                    contentDescription = "Foto de registro",
                    modifier = Modifier.size(120.dp),
                    contentScale = ContentScale.Crop
                )
            }

            Button(onClick = { pedirFoto() }) {
                Text(if (estado.fotoUri.isBlank()) "Tomar foto" else "Tomar otra foto")
            }
```

Parte 2, el resto de la columna y `cargarFoto`:

```
        OutlinedTextField(
            value = estado.nombre,
            onValueChange = viewModel::onNombreChange,
            label = { Text("Nombre completo") },
            isError = estado.errores.nombre != null,
            supportingText = {
                estado.errores.nombre?.let { mensaje ->
                    Text(mensaje)
                }
            },
            modifier = Modifier.fillMaxWidth()
        )

        OutlinedTextField(
            value = estado.correo,
            onValueChange = viewModel::onCorreoChange,
            label = { Text("Correo electrónico") },
            isError = estado.errores.correo != null,
            supportingText = {
                estado.errores.correo?.let { mensaje ->
                    Text(mensaje)
                }
            },
            modifier = Modifier.fillMaxWidth()
        )

        Row(verticalAlignment = Alignment.CenterVertically) {
            Checkbox(
                checked = estado.aceptaTerminos,
                onCheckedChange = viewModel::onAceptarTerminosChange
            )
            Text("Acepto los términos")
        }

        estado.errores.aceptaTerminos?.let { mensaje ->
            Text(mensaje)
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 16.dp))

        Text("Vista previa en tiempo real")
        Text("Nombre: ${estado.nombre}")
        Text("Correo: ${estado.correo}")
        Text("Términos aceptados: ${if (estado.aceptaTerminos) "Sí" else "No"}")

        Text("Intentos: $intentos")
        Text(if (puedeEnviar) "Listo para enviar" else "Faltan datos")

        Button(
            onClick = {
                intentos++
                if (viewModel.validar()) {
                    viewModel.guardar()
                } else {
                    scope.launch {
                        snackbarHostState.showSnackbar("Errores detectados. Revisa los
    campos.")
                    }
                }
            },
            enabled = !estado.guardando,
            colors = ButtonDefaults.buttonColors(containerColor = colorBoton)
        ) {
```

```
                if (estado.guardando) {
                    CircularProgressIndicator(
                        modifier = Modifier.size(20.dp),
                        color = Color.White,
                        strokeWidth = 2.dp
                    )
                } else {
                    Text("Registrarse")
                }
            }

            AnimatedVisibility(visible = estado.guardado) {
                Text("Datos guardados en el teléfono")
            }

            Button(
                onClick = { mainViewModel.navigateTo(Screen.Home) }
            ) {
                Text("Volver")
            }

            SnackbarHost(hostState = snackbarHostState)
        }
    }
}

private fun cargarFoto(contexto: Context, uri: String): ImageBitmap? {
    val opciones = BitmapFactory.Options().apply { inSampleSize = 4 }
    val bitmap = contexto.contentResolver.openInputStream(Uri.parse(uri))?.use { entrada ->
        BitmapFactory.decodeStream(entrada, null, opciones)
    } ?: return null
    return bitmap.asImageBitmap()
}
```

El import de `Arrangement` puede quedar sin uso al quitar `verticalArrangement`. Si Android Studio lo marca, bórralo.

Qué probar

1. Entra a registro. Debe verse **Tomar foto** y, al deslizar, el formulario de siempre.
2. Pulsa **Tomar foto**. La primera vez Android pide el permiso. Si lo rechazas, el aviso dice que sin permiso no se puede tomar la foto y el formulario sigue ahí.
3. Acepta el permiso, saca una foto y confírmala en la cámara. La imagen aparece sobre el nombre y el botón pasa a **Tomar otra foto**.
4. Rota el teléfono, de vertical a horizontal. La foto se mantiene.
5. Pulsa **Tomar otra foto** y cancela en la cámara. La foto anterior sigue en pantalla.
6. Llena nombre, correo y términos, y pulsa **Registrarse**. El guardado funciona igual, con o sin foto.